OPERATING SYSTEMS PROJECT REVIEW | CO: 1 TO CO: 5 COMPLETE COVERAGE | C99 / POSIX

Mini-Container Runtime

A Lightweight Linux Container Virtualization Engine Built from Scratch in C99
Utilizing Linux Namespaces, Unified Cgroups v2, and VFS Sandboxing

TEAM MEMBER 1

Tharakesha M

Reg No: 24BCE2538

B.Tech Computer Science & Engineering
Core Developer (Namespaces & Cgroups)

TEAM MEMBER 2

Mathu Metha S

Reg No: 24BCE2805

B.Tech Computer Science & Engineering
Core Developer (Concurrency & Testing)

GitHub: **github.com/THARAKESHA/mini-container**

Motivation: OS Virtualization vs. Hardware Emulation

Traditional Virtual Machines (Hypervisors)

- Virtualizes the physical **hardware layer** using a hypervisor (Type-1/2).
- Requires a dedicated, redundant guest OS kernel per instance.
- High memory footprint (1 - 4 GB) and slow boot latency (30 - 60s).
- Incurs hardware emulation translation penalties and CPU traps.
- Rigid, coarse-grained hardware resource partitioning.

OS-Level Containers (mini-container in C)

- Shares the single underlying host Linux kernel with zero emulation.
- Containers are regular OS processes wrapped in kernel-enforced limits.
- Sub-second boot times (10-20ms) and minimal RAM overhead (~5MB).
- Near-native bare-metal execution performance with hardware quotas.
- Fine-grained dynamic control via modern Linux Cgroups v2.

EFFICIENCY COMPARISON REPRESENTATION:

Boot Latency: **VM (45,000 ms)** vs. **Container (15 ms)** | Memory Overhead: **VM (~1500 MB)** vs. **Container (< 5 MB)** | CPU Overhead: **~0.1%**

Syllabus Alignment: Complete CO 1 to CO 5 Coverage

COURSE OUTCOMES MATRIX

Course Outcome (CO)	Syllabus Topics Covered	Implementation Mechanism in C
CO 1: OS Principles & Processes	System calls, User/Kernel modes, PCB, Process creation	`clone()` with isolated stack, `waitpid()`, custom PCB handling
CO 2: CPU Scheduling	CFS quotas, Preemption, Proportional Fair Share	cgroups `cpu.max` (40% quota) & `cpu.weight` (4:1 contest)
CO 3: Concurrency & IPC	Synchronization primitives, Semaphores, Shared Memory	POSIX `sem_t` (mutex, empty, full) & `mmap` ring buffer
CO 4: Memory Management	Virtual memory, Demand paging, Page faults, OOM Killer	cgroups `memory.max`, `pgfault` tracking (7,705 faults), `SIGKILL`
CO 5: Virtualization & Filesystems	Container virtualization, Mount propagation, VFS	Namespaces (`PID`, `UTS`, `Mount`, `IPC`), `chroot()`, private `/proc`

System Architecture & Execution Lifecycle

PROCESS LIFECYCLE FLOW

Coordinated Execution Pipeline

```
1. CLI Parsing:
   └─► sudo minicontainer run --memory 30M --cpu 40 /bin/sh

2. Host Parent Process:
   ├─► Allocates 1MB stack & creates UNIX sync pipe (barrier)
   ├─► clone(CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | ...)
   ├─► Creates /sys/fs/cgroup/minicontainer-[PID]
   ├─► Applies memory.max, cpu.max, pids.max
   ├─► Attaches child PID to cgroup.procs
   ├─► Signals child via pipe (unblocks execution)
   └─► Calls waitpid(), reads memory.stat, cleans cgroup

3. Container Child Process:
   ├─► Blocks on pipe until cgroups are attached
   ├─► sethostname("container-box")
   ├─► chroot() to rootfs & mounts private /proc
   └─► execvp(argv[0]) as PID 1
```

Architectural Safety Guarantees

- **Barrier Synchronization:** The child process pauses on `read(pipe[0])` until parent establishes cgroup limits, preventing race conditions.
- **Mount Privacy (MS_PRIVATE):** Prevents container mounts (`/proc`, `rootfs`) from leaking into the host operating system.
- **Automated Resource Reclamation:** Parent captures termination signals (OOM `SIGKILL`) via `waitpid()` and removes temporary cgroups.
- **Zero-Dependency Engineering:** Built with pure C99 and native Linux system calls without third-party runtime frameworks.

Core Isolation Subsystems: The Three Pillars

KERNEL SUBSYSTEMS

1. Namespaces

What the process can **SEE**

- `CLONE_NEWPID`: Child becomes PID 1; host processes hidden.
- `CLONE_NEWUTS`: Isolated hostname without altering host.
- `CLONE_NEWNS`: Decouples VFS mount table.
- `CLONE_NEWIPC`: Separate POSIX message queues.

2. Cgroups v2

What the process can **USE**

- `memory.max`: Strict RAM ceiling; triggers OOM Killer.
- `cpu.max`: CFS scheduler bandwidth quota.
- `cpu.weight`: Proportional fair-share scheduling.
- `pids.max`: Prevents fork bomb attacks.

3. Rootfs Sandboxing

Where the process can **TOUCH**

- **Alpine Linux Rootfs:** Lightweight ~3.5MB userland.
- `chroot()`: Cages process inside directory tree.
- **Private /proc:** Isolated view of PID 1.
- Prevents traversal into host root filesystems.

Classical Concurrency & IPC Synchronization (CO: 3)

CONCURRENCY PRIMITIVES

Producer-Consumer with Semaphores (sem_t)

- **Bounded-Buffer Concurrency Problem** implemented inside the container sandbox.
- **Binary Semaphore (mutex):** Initialized to 1; enforces mutual exclusion for the critical section.
- **Counting Semaphore (empty_slots):** Initialized to buffer capacity (N = 5).
- **Counting Semaphore (full_slots):** Initialized to 0 items available.
- **Shared Memory (mmap):** Allocated via `MAP_SHARED | MAP_ANONYMOUS` across processes.
- **Result:** All 10 operations exchanged with zero race conditions or buffer corruption.

Parent-Child IPC Pipe Synchronization

- **The Race Condition Hazard:** Cloned child process could execute payload before parent attaches cgroups limits.
- **The POSIX Pipe Solution:** Parent and child share a unidirectional `sync_pipe[2]`.
- **Child Execution:** Calls `read(sync_pipe[0])` and immediately **BLOCKS**.
- **Parent Execution:** Completes cgroup setup, then writes single trigger byte `'G'`.
- **Unblocking:** Child receives byte, closes pipe, and proceeds safely to `execvp()`.
- Ensures deterministic, race-free resource limits from microsecond zero.

Empirical Laboratory Benchmarks & Verification

OBSERVED METRICS

1. Process Virtualization (CO: 1 & 5)

- Container Process reports **PID = 1**; Parent PID = 0.
- Host operating system processes (47 active) are completely hidden.
- Hostname virtualized to `isolated-box` without altering host.

2. Memory Ceiling & Demand Paging (CO: 4)

- Workload allocated 30MB of 60MB; terminated by **SIGKILL (signal 9)**.
- Kernel logged **7,705 Minor Page Faults**, validating Demand Paging.
- 1 Major Page Fault; Swap disabled to force immediate OOM protection.

3. Proportional Fair-Share CPU Contest (CO: 2)

- Competing containers on CPU Core 0 with CFS weights 800 vs 200.
- Container A CPU Time: **2.78s** | Container B CPU Time: **0.69s**.
- Exact **4.02 : 1** execution ratio (matches theoretical 4:1 with 99.5% accuracy!).

4. Fork Bomb & PCB Table Defense (CO: 1 & 3)

- Stress test attempted to spawn 50 concurrent processes.
- Cgroups `pids.max = 10` capped execution; Children 1 - 9 spawned.
- 10th fork() blocked with **errno 11 (EAGAIN)**; host OS remained responsive.

EVALUATION & LIVE DEMONSTRATION

Interactive Project Demonstration

Complete C99 Runtime Covering CO: 1 to CO: 5 with Master Interactive Menu and Real-Time Kernel Metrics

```
$ sudo bash interactive_menu.sh
```

(Or double-click **Run-MiniContainer.bat** directly on your Windows Desktop)

Project Team: Tharakesha M (24BCE2538) & Mathu Metha S (24BCE2805)

GitHub: https://github.com/THARAKESHA/mini-container